Name: K. Sai Kumar Reddy

Batch-42

2303a52223

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions

while preserving the output.

Prompt:

Review the legacy code and values using an explicit approach and use an Ai tool for refactoring the code and values to make it more efficient and maintainable.

```
# Legacy Code
 values = [2, 4, 6, 8, 10]
doubled = []
 for v in values:
        doubled.append(v * 2)
 print(doubled)
```

Code:

```
# Refactored Code using List Comprehension
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
(.venv) PS C:\Users\ksair\Download
nloads/Competitive Programing/Assi
[4, 8, 12, 16, 20]
```

Justification:

The refactored code uses a list comprehension, which is more concise and readable than the traditional for loop. It eliminates the need for an explicit loop and the intermediate list appending, making the code more efficient and easier to maintain.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word

by word.

Task Description

• Analyze the legacy code that constructs a sentence using repeated string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output unchanged.

Prompt:

Analyze the legacy code that constructs a sentence using repeated string concatenation. Ask AI to suggest a more efficient and readable approach.

words = ["Refactoring", "with", "AI", "improves", "quality"]

message = ""

for w in words:

message += w + " "

print(message.strip())

Code:

```
# Legacy Code
# words = ["Refactoring", "with", "AI", "improves",
# "quality"]
# message = ""
# for w in words:
# message += w + " "
# print(message.strip())
# code:
# Improved Code
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> & "c:/Users/ksair/D
nloads/Competitive Programing/Assignment-13.4.py"
Refactoring with AI improves quality
```

Justification:

The improved code uses the join() method, which is more efficient than repeated string concatenation. It also enhances readability by clearly indicating that we are joining the words with a space. The strip() method is no longer necessary since join() does not add an extra space at the end.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary access methods.

• Ensure the behavior remains the same for missing keys.

Prompt:

You have a piece of legacy code that checks for the presence of keys in a dictionary before accessing their values. The code looks something like this:

Legacy Code

config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"])

else:

print("Default timeout used")

Code:

```
#  You have a piece of legacy code that checks for the presence of keys in a dictionary before accessing their values. The co
# Legacy Code
# config = {"host": "localhost", "port": 8080}
# if "timeout" in config:
# print(config["timeout"])
# else:
# print("Default timeout used")

# code;
config = {"host": "localhost", "port": 8080}
timeout = config.get("timeout", "Default timeout used")
print(timeout)
```

Output:

```
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> & "c:/Users/ksair/Downloads/Com
nloads/Competitive Programing/Assignment-13.4.py"
Default timeout used
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> & "c:\Users\ksair\Downloads\Com
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing>
```

Justification:

The original code checks for the presence of the "timeout" key in the config dictionary and prints its value if it exists. If the key is not present, it prints a default message. The refactored code uses the get() method of the dictionary, which allows us to specify a default value to return if the key is not found. This eliminates the need for an explicit if-else statement, making the code more concise and easier to read while maintaining the same functionality.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving functionality.

Prompt:

Multiply if-elif cases in the code. The Ai to suggest a cleaner ,scalable alternative and refactor the code accordingly using mapping technique.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
elif action == "subtract":
result = x - y
elif action == "multiply":
result = x * y
elif action == "divide":
result = x / y
else:
```

result = None

print(result)

Code:

```
# x, y = 10, 2
# if action == "add":
# result = x + y
# elif action == "subtract":
# result = x - y
# elif action == "multiply":
# result = x * y
# elif action == "divide":
# result = x / y
# else:
# result = None
# print(result)
# Refactored Code using Mapping Technique
action = "divide"
x, y = 10, 2
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    return a / b
operations = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply,
    "divide": divide
}
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

Output:

```
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> &
nloads/Competitive Programing/Assignment-13.4.py"
5.0
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing>
```

Justification:

The refactored code uses a mapping technique to associate each action with its corresponding function. This approach eliminates the need for multiple if-elif statements, making the code cleaner and more scalable. Adding new operations in the future would simply involve defining a new function and adding it to the operations dictionary, without modifying existing code.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops

to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and

readable form.

• Maintain the same output behavior.

Prompt:

Find the item in the inventory list and return its index.""Refactor the search logic to use a more concise approach, such as list comprehensions or built-in functions."

```
inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:

if item == "eraser":

found = True

break

print("Item Available" if found else "Item Not Available")
```

Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")

nventory = ["pen", "notebook", "eraser", "marker"]
tem_to_find = "eraser"
ndex = inventory.index(item_to_find) if item_to_find in inventory else -1
rint(f"Item {'Available at index ' + str(index) if index != -1 else 'Not Available'}")
```

Output:

```
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> &
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing> &
graming/Assignment-13.4.py"
Item Available at index 2
(.venv) PS C:\Users\ksair\Downloads\Competitive Programing>
```

Justification:

This refactored code uses the built-in `index()` method to find the index of the item directly, and a conditional expression to check if the item exists in the inventory. This approach is more concise and eliminates the need for a loop and a separate flag variable.